University of Windsor

01 Course Overview & Modern Database Architecture Part II

Dr. Kalyani Selvarajah
School of Computer Science
University of Windsor

Advanced Database Topics
COMP 8157 01
Winter 2026

IDS

Honeywell - The first network DBMS.

Charles Bachman

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

SQL SERVER

MySQL

PostgreSQL

SQLite

Microsoft SQL Server

MySQL

PostgreSQL

SQLite

IDS
Honeywell - The first network DBMS.
Charles Bachman

OBJECT-ORIENTED DATABASES
In 1983

DB2
IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

Data Warehouses
Distributed / Shared-Nothing
Netezza, ParAccel
MonetDb, Greenplum
Datallegro, Vertica

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS
Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS
IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

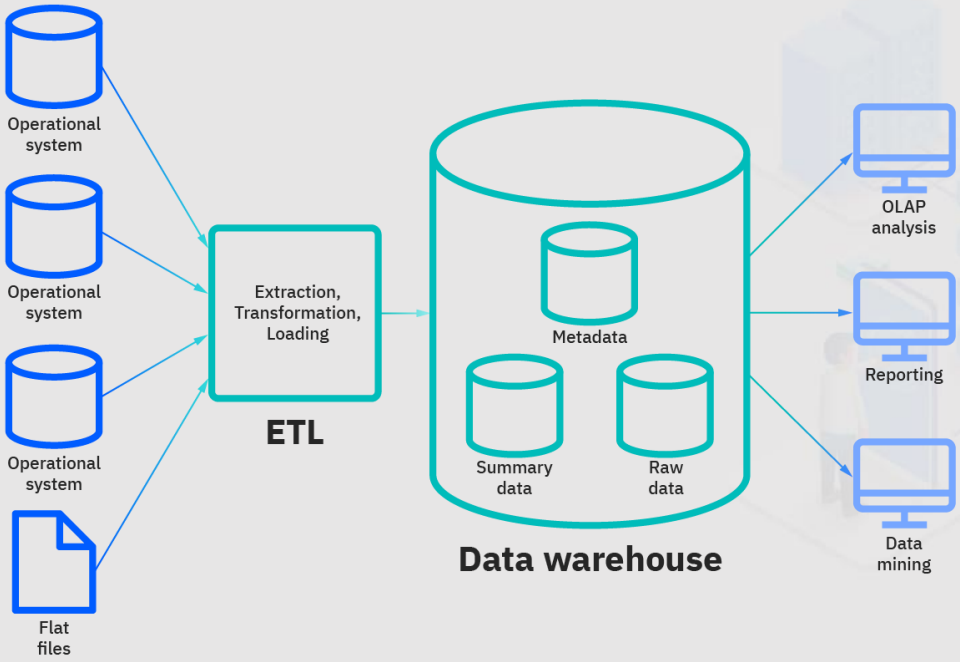
Relational Model
IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, INGRES, Oracle

SQL SERVER
MySQL
PostgreSQL
SQLite

Data Warehouses

- A data warehouse was deemed the solution to meet the requirements of a system capable of supporting decision making, receiving data from multiple operational data sources.



Late 2000s Challenges

By the mid-to-late 2000s, traditional relational data warehouses (Teradata, Netezza, Vertica, Greenplum, etc.) faced new challenges:

1. **Explosion of data volume and variety**
   - Web companies like Google, Facebook, Amazon, and Twitter generated massive, **high-velocity, semi-structured or unstructured data** (logs, clickstreams, social media, JSON, XML).
2. **Scaling limitations of RDBMS**
   - Vertical scaling (bigger servers) was expensive.
   - Traditional RDBMS struggled with **horizontal scaling across distributed commodity hardware**.
3. **Flexibility requirements**
   - Fixed relational schemas were too rigid for rapidly changing data and new applications.

The Rise of the NoSQL Movement

- “NoSQL” (Not Only SQL) emerged as a response to these (slide 6 challenges:
- Databases that eschew traditional relational constraints for horizontal scalability, schema flexibility, and high performance in certain workloads.
- **Core motivations:**
    1. **Scalability:** Designed for distributed, shared-nothing architectures.
    2. **Schema flexibility:** Can handle JSON, XML, key-value pairs without rigid tables.
    3. **High availability:** Often favor eventual consistency over strict ACID.
    4. **Specialized workloads:** Optimized for analytics, search, or graph traversal rather than transactional integrity.

1960

IDS

Honeywell - The first network DBMS.

Charles Bachman

1970

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

1980

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1990

SQL SERVER

MySQL

PostgreSQL

SQLite

2000

DDBMS

NoSQL

MongoDB,
Cassandra
HBase
Oracle NoSQL

Data Warehouses

Distributed / Shared-Nothing

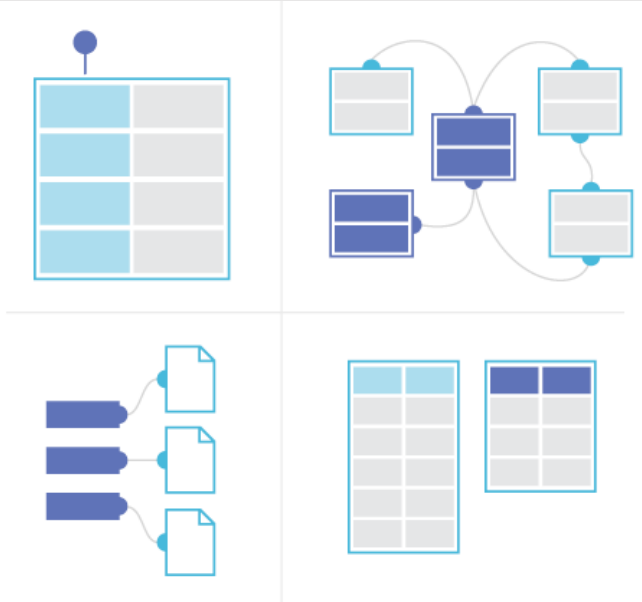
Netezza, ParAccel
MonetDb, Greenplum
Datallegro, Vertica

2010

2020

Key NoSQL Database Types (Late 2000s)

Type	Example	How it works
Key-Value Stores	Redis, Riak, Dynamo	Data stored as a simple key → value pair (like a giant dictionary or hashmap).
Document Stores	MongoDB, CouchDB	Data stored as documents (JSON-like structures) that can vary in shape.
Column-Oriented Stores	Cassandra, HBase	Data stored in rows, but grouped into column families. Similar to RDBMS tables, but more flexible — each row can have different columns.
Graph Databases	Neo4j	Data stored as nodes (entities) and edges (relationships), optimized for traversing connections.



Nosql Type With Real-world Company Examples

1. **Key-Value Stores:**

   Examples: Redis, Amazon DynamoDB, Riak

   - ✓ **Amazon:** Uses **DynamoDB** (their own key-value + document store) to power **shopping carts, product catalogs, and session management** across millions of users.
   - ✓ **Twitter:** Uses **Redis** as a caching layer for tweets and timelines so users get near-instant feed refreshes.
   - ✓ **Uber:** Uses **Riak** (historically) and Redis to handle **geolocation lookups** (which driver is closest?).

   Key-value = ultra-fast lookups for high-traffic apps.

2. **Document Stores:**

   Examples: MongoDB, CouchDB

   - ✓ **eBay:** Uses **MongoDB** for storing product metadata, which varies widely (different sellers, product attributes).
   - ✓ **Forbes & The Guardian:** Use **MongoDB** as a **content management system** backend for storing articles with different layouts and fields.
   - ✓ **LinkedIn (some subsystems):** Uses document stores for **member profiles**, which may have very different sets of attributes.

   Document DB = perfect for flexible, JSON-style data like product catalogs, user profiles, or articles.

NoSQL Type With Real-world Company Examples

3. **Column-Family (Wide-Column) Stores:**

   Examples: Cassandra, HBase

   - ✓ **Facebook:** Built **Cassandra** (later open-sourced) to handle **Inbox Search** (huge write-heavy workload with billions of messages).
   - ✓ **Netflix:** Uses **Cassandra** for **streaming activity logs** and **recommendation engines**, handling millions of writes per second globally.
   - ✓ **Spotify:** Uses **Cassandra** to store **user listening histories** for recommendations.

   Column-family = great for big, write-heavy, time-series style data at global scale.

4. **Graph Databases:**

   Examples: Neo4j, Amazon Neptune, TigerGraph

   - ✓ **Facebook:** Uses graph concepts (internally, a custom graph DB system) for its **social graph** (friends, likes, relationships).
   - ✓ **PayPal:** Uses graph databases for **fraud detection** by identifying suspicious transaction patterns across connected accounts.
   - ✓ **Airbnb:** Uses graph DBs to improve **search and recommendation systems** (e.g., which listings are most relevant based on traveler history and preferences).

   Graph DB = when relationships (who's connected to who/what) are the key.

How NoSQL Changed the Landscape

Complemented data warehouses (DW + NoSQL together): Warehouses still handled structured historical analytics; NoSQL handled **real-time, high-volume, semi-structured data**.

Workflow example:

- ✓ NoSQL ingests and stores real-time clickstream data from a website.
- ✓ This fresh data is streamed into dashboards for immediate monitoring.
- ✓ Periodically, aggregated or curated subsets of that data move into the data warehouse for historical trend analysis.

Enabled "Big Data" analytics: Scalable storage and distributed computing (e.g., Hadoop ecosystem).

Promoted polyglot persistence: Systems now combine multiple databases optimized for different workloads (RDBMS + NoSQL + warehouses).

Relational Table vs. JSON

Relational table

id	name	email	age	city	phone
1	Alice	alice@email.com	29	Toronto	555-1234
2	Bob	bob@email.com			

- ✓ Rigid, consistent structure.
- ✓ Great when every user has the same fields.
- ✓ Adding new fields requires schema migrations or extra tables.

JSON (Document)

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@email.com",
  "age": 29,
  "city": "Toronto",
  "phone": "555-1234",
  "twitter": "@aliceT"
}

{
  "id": 2,
  "name": "Bob",
  "email": "bob@email.com",
  "favoriteColor": "blue"
}
```

- ✓ Flexible, schema-less.
- ✓ Best when attributes vary widely across users.
- ✓ No schema migration required — just add new key-value pairs.

?

Which suits a user profile with variable attributes?

A **JSON (document-based)** model suits a **user profile with variable attributes** much better, because it allows flexibility (different users can have different sets of fields) without constant schema changes.

1960

IDS

Honeywell - The first network DBMS.

Charles Bachman

1970

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

1980

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1990

SQL SERVER

MySQL

PostgreSQL

SQLite

2000

DDBMS

NoSQL

MongoDB,
Cassandra
HBase
Oracle NoSQL

Data Warehouses

Distributed / Shared-Nothing

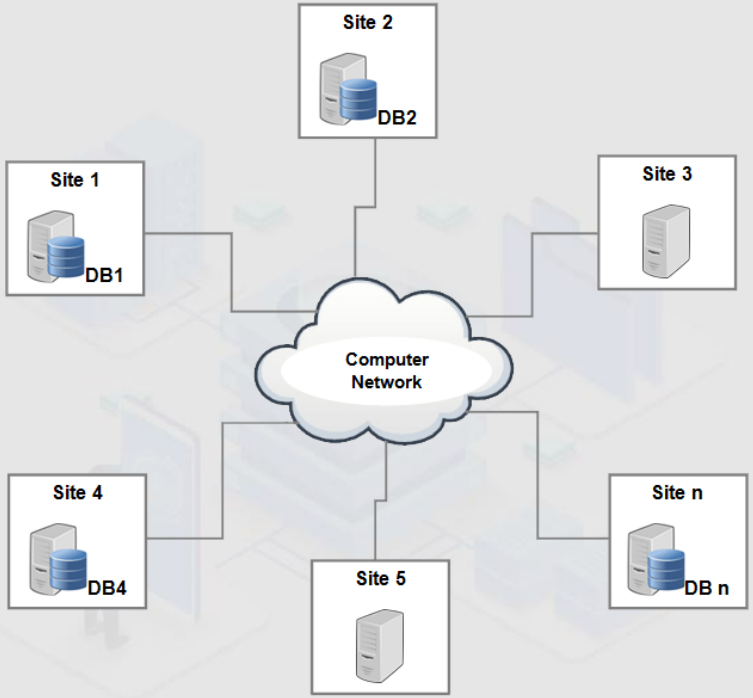
Netezza, ParAccel
MonetDb, Greenplum
Datallegro, Vertica

2010

2020

Distributed DBMS

- A distributed database is a logically interrelated collection of shared data (and a description of this data), physically distributed over a computer network.
- A distributed DBMS is the software system that permits the management of the distributed database and makes the distribution transparent to users.
- A DDBMS consists of a single logical database split into a number of fragments.
- Each fragment is stored on one or more computers (replicas) under the control of a separate DBMS, with the computers connected by a network.
- Each site is capable of independently processing user requests that require access to local data (that is, each site has some degree of local autonomy) and is also capable of processing data stored on other computers in the network.



Why NoSQL → Distributed DBMS?

1. **Data Explosion**
   - ✓ Petabytes of data from web, IoT, logs, social media.
   - ✓ A single server can't store or process it → distribute across many machines.

2. **High Availability**
   - ✓ Applications (e.g., Amazon, Netflix, Facebook) can't go down.
   - ✓ Distributed systems replicate data across nodes & regions.

3. **Performance at Scale**
   - ✓ Local queries are fast, but global apps need millions of reads/writes per second.
   - ✓ Achieved via **sharding + replication**.

4. **Flexibility of Modern Apps**
   - ✓ Cloud-native apps expect elastic growth.
   - ✓ Distributed DBMS auto-scales with more nodes.

NoSQL (late 2000s) emerged to solve **scalability + flexibility** problems that relational warehouses struggled with. Almost all successful NoSQL systems (Cassandra, DynamoDB, HBase, MongoDB clusters) are **distributed by design**.

NoSQL showed how to scale and stay flexible, but distributed DBMS generalized the idea: apply distribution to any data model (relational or not), while balancing the CAP theorem trade-offs.

CAP Theorem

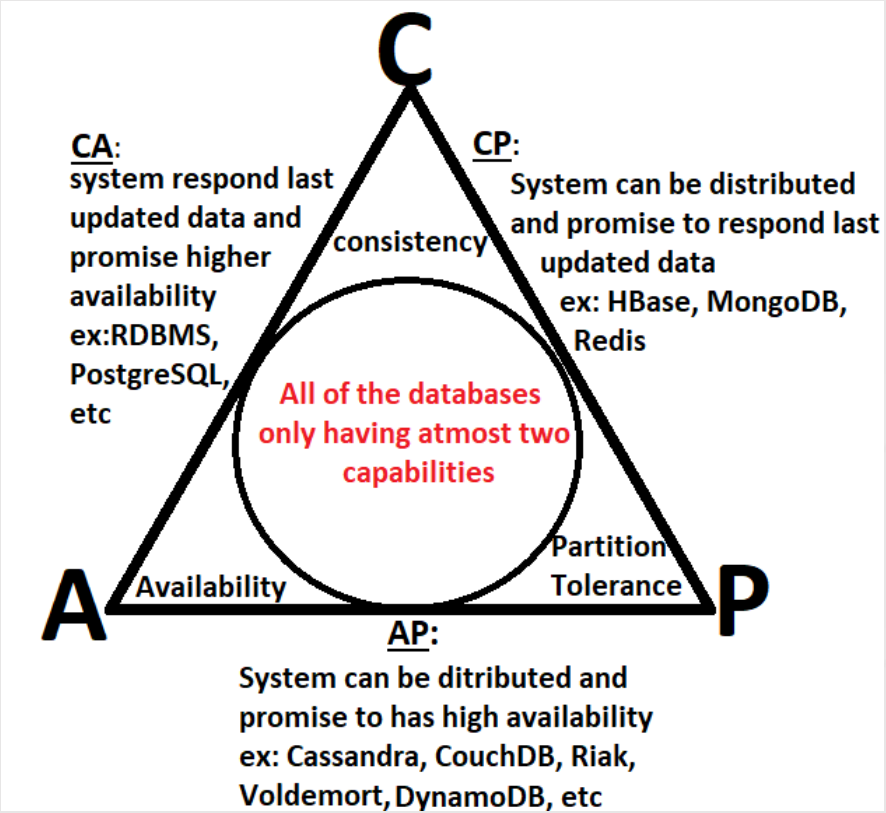
The CAP theorem, also known as **Brewer's theorem**, states that any distributed data store can only guarantee two out of three desirable properties:

C – Consistency: Every user sees the same data at the same time.

A – Availability: The system always responds, even if some parts fail.

P – Partition Tolerance: The system keeps working even if network links break between servers.

CP = correctness first
AP = availability first
CA = only possible in non-distributed systems



Source: https://www.geeksforgeeks.org/dbms/the-cap-theorem-in-dbms/

1960

IDS

Honeywell - The first network DBMS.

Charles Bachman

1970

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

1980

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1990

SQL SERVER

MySQL

PostgreSQL

SQLite

2000

DDBMS

NoSQL

MongoDB
Cassandra
HBase
Oracle NoSQL

DynamoDB,
Bigtable,
CockroachDB

Data Warehouses

Distributed / Shared-Nothing

Netezza, ParAccel
MonetDb, Greenplum
Datallegro, Vertica

2010

NewSQL

a relational database with the scalable properties of NoSQL

VoltDB, Clustrix, MemSQL, Spanner

2020

VOLTDB

Clustrix

memSQL

Cloud Spanner

NewSQL databases

The necessity of NewSQL databases arises from the limitations of traditional RDBMS and NoSQL systems when handling modern application requirements. NewSQL emerged to bridge the gap between RDBMS and NoSQL:

Feature	RDBMS	NoSQL	NewSQL
ACID Transactions	✓	✗/partial	✓
SQL Support	✓	✗/limited	✓
Horizontal Scalability	✗	✓	✓
High Throughput	✗	✓	✓
Flexible Schema	✗	✓	✓(some systems)

Goal: Bring back SQL + ACID guarantees, but with NoSQL-like scalability (horizontal scaling, distributed architecture).

While NewSQL excels at OLTP, it isn't always the best for analytical workloads (OLAP) or for handling semi-structured / unstructured data like JSON, logs, or graph-like relationships.

1960

IDS

Honeywell - The first network DBMS.
Charles Bachman

1970

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

1980

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1990

SQL SERVER
MySQL
PostgreSQL
SQLite

2000

DDBMS
NoSQL

MongoDB
Cassandra
HBase
Oracle NoSQL

Data Warehouses

Distributed / Shared-Nothing

Netezza, ParAccel
MonetDb, Greenplum
Datallegro, Vertica

2010

NewSQL

a relational database with the scalable properties of NoSQL
VoltDB, Clustrix, MemSQL, Spanner

Hybrid Systems

Execute fast OLTP like a NewSQL system while also executing complex OLAP queries like a data warehouse system.
Hyper, snappy, JustOne

2020

Hybrid Systems :
(HTAP (Hybrid Transactional/Analytical Processing))

A **Hybrid system** = a database that combines **multiple data models and workloads** (transactional + analytical + flexible data types) in **one unified platform**.

Goal: Avoid having **separate databases** for OLTP, OLAP, document, graph, etc.

Why Hybrid is Necessary Over NewSQL?

- ✓ **Mixed Workloads (OLTP + OLAP)**
- ✓ **Multi-Model Data**
- ✓ **Reduced Complexity**
- ✓ **Cloud-Native Expectations**

NewSQL = SQL + ACID + distributed scaling → best for distributed OLTP.

Hybrid DBMS (HTAP, multi-model) = SQL + NoSQL + OLTP + OLAP in one

NewSQL solved “scale with SQL,” but Hybrid systems solve “everything in one system.”

Polyglot Persistence Architecture

Definition: Using **different databases, each specialized for a specific workload or data model**, within the same application.

Philosophy: *“Use the right tool for the right job.”*

Example:

- ✓ Use **MongoDB** for flexible user profiles (documents).
- ✓ Use **Neo4j** for social graph relationships.
- ✓ Use **Postgres** for transactional orders (relational).
- ✓ Use **Cassandra** for large-scale event logs (wide-column).

Pros: Best performance and features for each use case.

Cons: Complex! You now manage multiple systems, data pipelines, consistency issues, and integration logic.

Hybrid systems vs Polyglot Persistence.

Polyglot Persistence: Multiple specialized DBs → more flexibility, but more complexity.

Hybrid Systems: One DB that tries to cover many roles → less complexity, but sometimes **compromises on specialization**.

?

Which is best in terms of industry?

Polyglot Persistence = Best for **hyperscale companies (FAANG-level)** that can afford dozens of specialized systems and teams to glue them together.
Hybrid Systems = Best for **most enterprises & startups** — one system that simplifies data architecture, even if it's not perfectly optimized for every workload.

1960

IDS

Honeywell - The first network DBMS.
Charles Bachman

1970

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

1980

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1990

SQL SERVER
MySQL
PostgreSQL
SQLite

2000

DDBMS
NoSQL

MongoDB
Cassandra
HBase
Oracle NoSQL

Data Warehouses

Distributed / Shared-Nothing

Netezza, ParAccel
MonetDb, Greenplum
Datallegro, Vertica

2010

NewSQL

a relational database with the scalable properties of NoSQL
VoltDB, Clustrix, MemSQL, Spanner

Hybrid Systems

Execute fast OLTP like a NewSQL system while also executing complex OLAP queries like a data warehouse system.
Hyper, nappy, JustOne

Cloud System

First database-as-a-service (DBaaS) offerings were "containerized" versions of existing DBMSs.
RedShift &Aurora from Amazon, Snowflake, Xeround

2020

aws
Microsoft Azure
Google Cloud Platform
snowflake

Cloud-Native Database Architectures

Designed **from the ground up** for **elastic, distributed cloud environments** (not just “lift and shift” of an old DB to AWS/Azure/GCP).

Core principles:

- ✓ **Elastic scaling** (scale out/in automatically).
- ✓ **High availability & fault tolerance** across regions.
- ✓ **Separation of storage and compute** → cheaper, flexible resource use.
- ✓ **Serverless options** → pay only for what you use.
- ✓ **API-driven + managed services** → less operational burden (less manual work)

Why Cloud-Native Matters?

1. **Old DBs (RDBMS, traditional DW):**
   - ✓ Hard to scale, designed for single-node servers, limited fault tolerance.
2. **Cloud-Native DBs:**
   - ✓ Solve **scale + reliability + cost efficiency** by using cloud primitives (object storage, containers, service meshes).
3. **For industry:**
   - ✓ Startups use them to avoid operational teams.
   - ✓ Enterprises use them for agility (global rollouts, multi-region apps).
   - ✓ Big Tech uses them for cross-region consistency (Spanner for Google Ads, Gmail).

Cloud-Native Database Architectures

Examples in Practice:

- ✓ **Netflix** → heavy use of **Cassandra on AWS**, plus Snowflake/Redshift for analytics.
- ✓ **Facebook/Meta** → building their own, but many features (disaggregation, global consistency) mirror cloud-native DBs.
- ✓ **Fintechs** → often adopt **CockroachDB / YugabyteDB** for global banking transactions.

Cloud-native DBs are the evolution beyond RDBMS, NoSQL, and NewSQL — they exploit the cloud's elasticity, global reach, and storage/compute separation. They let businesses scale like Google or Netflix *without building the infrastructure themselves*.

What Problem Did Each Stage Solve?

- ✓ **RDBMS (1960s–1980s):** Structured data + ACID, but limited scalability.
- ✓ **NoSQL (Late 2000s):** Horizontal scale + schema flexibility, but weaker consistency.
- ✓ **NewSQL (2010s):** SQL + ACID + scalability, but focused mainly on OLTP.
- ✓ **Hybrid (2010s–2020s):** Unified OLTP + OLAP + multi-model, but still heavy to operate.
- ✓ **Cloud-Native DBs (2020s–):** Elastic scale, global availability, serverless, storage/compute separation.

(Optional) Specialized Data Bases

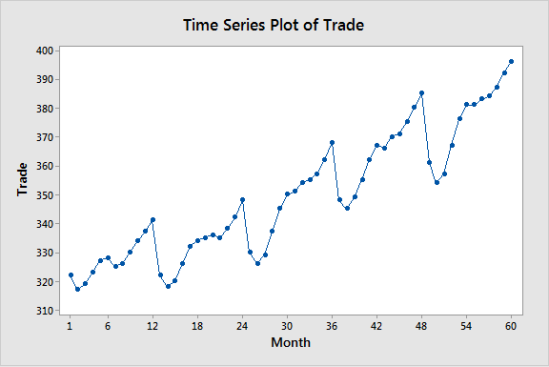
1. **Time-Series Databases (TSDBs)**
   - ✓ **Purpose:** Optimized for data that arrives as **time-stamped events**.
   - ✓ **Use Cases:** IoT sensors, stock trading, server monitoring (metrics).
   - ✓ **Examples:** InfluxDB, TimescaleDB (Postgres extension), Prometheus.

2. **Graph Databases**
   - ✓ **Purpose:** Store and query data as **nodes and edges** (relationships first).
   - ✓ **Use Cases:** Social networks, fraud detection, recommendation engines.
   - ✓ **Examples:** Neo4j, TigerGraph, Amazon Neptune.

3. **Vector Databases (AI/ML-focused, Emerging)**
   - ✓ **Purpose:** Store high-dimensional vectors for similarity search.
   - ✓ **Use Cases:** Generative AI, semantic search, image recognition.
   - ✓ **Examples:** Pinecone, Weaviate, Milvus, FAISS.

Time-Series Databases (TSDBs)

- A Time Series Database is a database optimized for storing, retrieving, and analyzing data that is ***indexed by time.***



- **Key Characteristics of Time-Series Data:**

  Always associated with a **timestamp**

  Data streams are **continuous and structured**

  Often **write-heavy**, with periodic aggregation

  Trend focus: patterns over time matter more than individual records

  Can apply **retention policies** to purge old data.

(temperature = 22.5) ← meaningless
(temperature = 22.5 at 10:01:05) ← useful

Time-Series Databases (TSDBs)

Common Use Cases:

- ✓ **IoT & Industrial Monitoring** — sensors, machines, connected devices produce constant metrics.
- ✓ **Infrastructure & Operations Metrics** — server, network, and application performance data.
- ✓ **Predictive Maintenance & Anomaly Detection** — identify issues before failures occur.
- ✓ **Finance & Market Data Analysis** — analyze price changes, trends over time.
- ✓ **Real-Time Alerts & Dashboards** — trigger alarms based on threshold breaches.

Specialized Platforms & Databases:

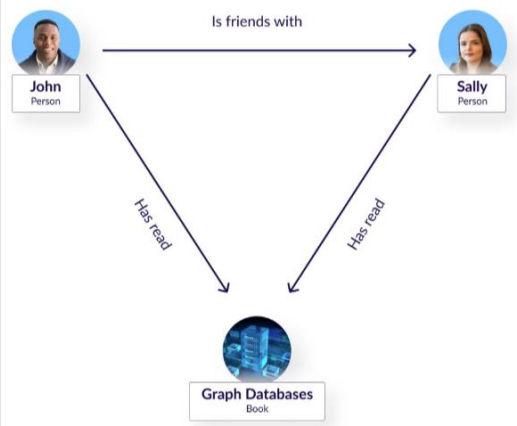
InfluxDB: One of the most widely used standalone TSDBs with high ingestion rates, built-in time-series functions, and retention policies.
Prometheus: Popular for real-time monitoring & alerting in DevOps and cloud environments (with its own PromQL query language).
TimescaleDB: An extension for PostgreSQL that optimizes it for time-series workloads

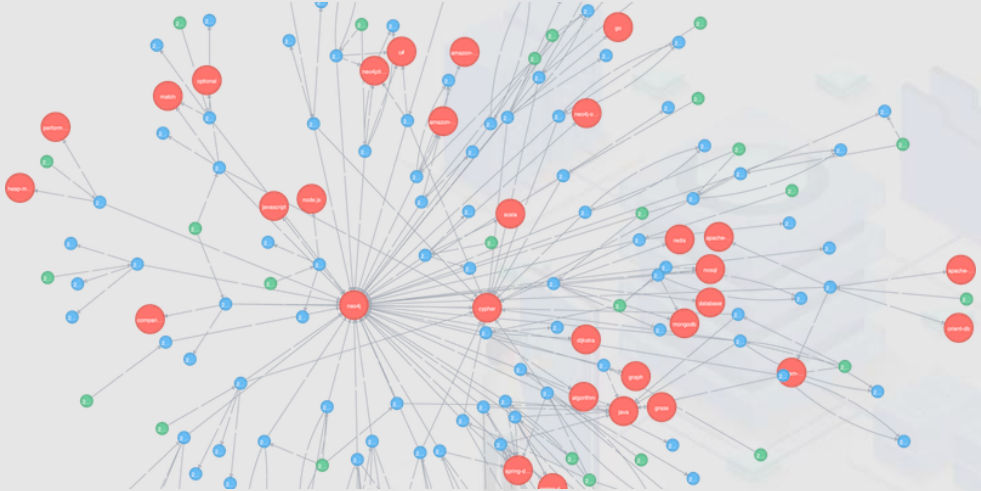
Data Ingestion

Apache Kafka: A high-throughput event streaming platform often used as the backbone for real-time data pipelines.

Graph Databases

A graph database is defined as a specialized, single-purpose platform for creating and manipulating graphs. Graphs contain nodes, edges, and properties, all of which are used to represent and store data in a way that relational databases are not equipped to do.

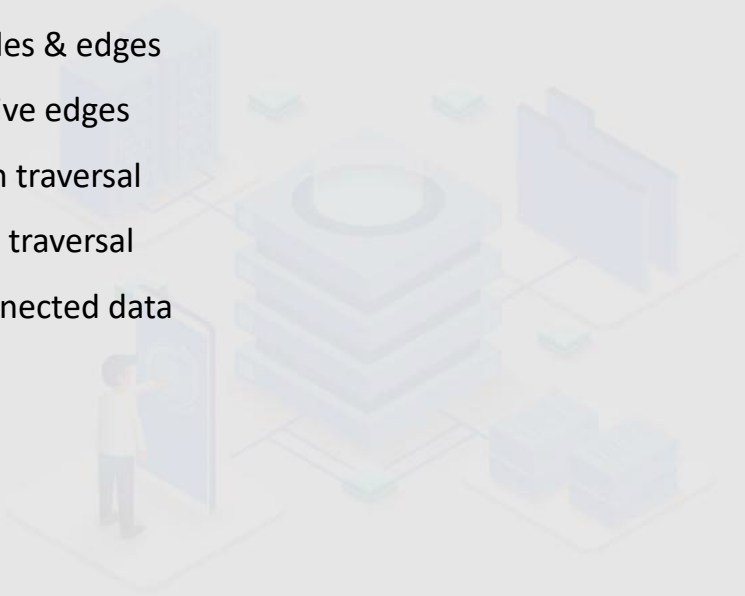




Source: https://aerospike.com/blog/what-is-behind-the-surging-interest-in-graph-databases/

How Graph Databases differ from relational databases

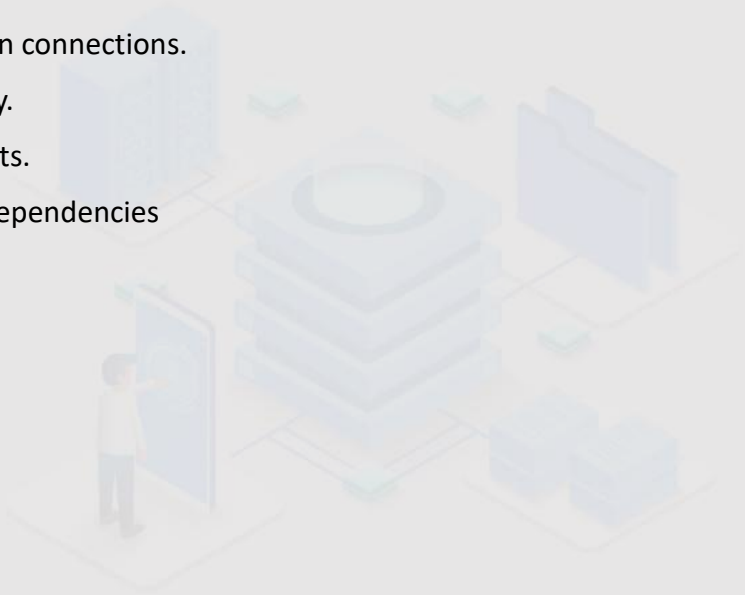
Feature	Relational DB	Graph DB
Data model	Tables	Nodes & edges
Relationships	Foreign keys + joins	Native edges
Query pattern	JOIN-heavy	Path traversal
Performance	Slows with joins	Fast traversal
Best for	Transactions	Connected data



Graph Databases

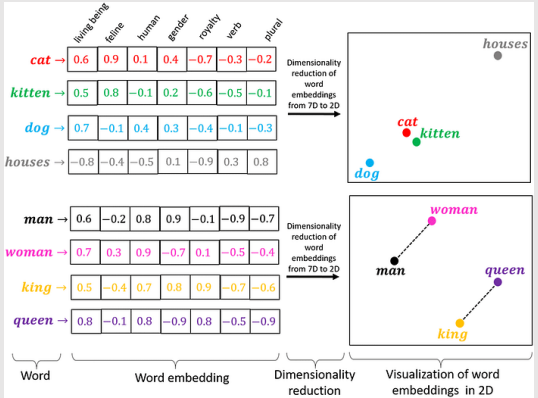
Common Use Cases

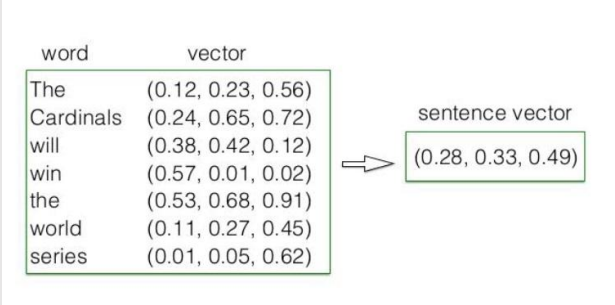
- ✓ **Social Networks:** Finding friends of friends.
- ✓ **Recommendation Engines:** Suggesting products or content based on connections.
- ✓ **Fraud Detection:** Identifying suspicious patterns and rings of activity.
- ✓ **Knowledge Graphs:** Connecting disparate information to find insights.
- ✓ **Network/IT Management:** Understanding complex infrastructure dependencies



Vector Databases

- A vector database is a special type of database designed to store and search data that has been converted into numerical vectors (lists of numbers). Modern AI systems (like image recognition, speech, or ChatGPT-style models) convert things such as text, images, audio, or video into vectors.
- Each vector captures the meaning or features of that item in many dimensions (often hundreds or thousands).
  - For example:
    - A sentence → a vector of 768 numbers
    - An image → a vector describing shapes, colors, objects
    - A video → a vector describing motion, scenes, etc.
  - These are called **high-dimensional vectors** because they have many dimensions (many numbers).





Source: https://medium.com/@hari4om/word-embedding-d816f643140

Vector Databases

A vector database may store millions or billions of vectors, each with hundreds or thousands of dimensions. That is huge. Managing this efficiently requires very different techniques from traditional databases.

1. They do NOT scan all vectors: vector databases use **Approximate Nearest Neighbor (ANN) indexing**.
2. They use special index structures:
   - ✓ Graph-based indexes (HNSW)- Used by FAISS, Milvus, Pinecone, Weaviate.
   - ✓ Clustering (IVF – Inverted File Index)
3. They separate storage from search
4. They are distributed systems
5. They use GPUs and SIMD
6. They update indexes incrementally

Discussion Questions

1. If you were designing a new social media platform today, would you choose RDBMS, NoSQL, or a hybrid? Why?
2. Do you think the “one database to rule them all” approach (hybrid systems) is realistic, or will polyglot persistence always be necessary?
3. For industries like banking or healthcare, where consistency is critical, which database family makes the most sense?
4. Why do you think cloud-native databases are growing so fast, even when many companies already use RDBMS/NoSQL successfully?
5. If you were Netflix (millions of users + global scale), which database approach would you prioritize and why?
6. What trade-offs do you notice between scalability, consistency, and operational complexity across DB generations?

Next Lecture

Introduction to Bigdata